CSC3043S 2026 — Assignment 1

Training a Transformer Language Model

Due: 19 August 2026, 23:59 | **Total marks: 60**

Late policy. Submissions up to 24 hours late (i.e. until 23:59 on Thursday 20 August) are accepted with a 10% penalty applied to the assignment mark. No submissions are accepted after that.

---

1. Overview

In Tutorial 1 you implemented a character-level BPE tokenizer and a modern dense Transformer language model, and trained it with plain SGD on 5,000 TinyStories documents. That was a *minimum viable* language model: the tokenizer was quadratic, the optimiser had no schedule and no clipping, everything ran in fp32, generation re-ran the whole context at every step, and the training set was small enough to fit in a notebook.

In this assignment you turn that into a full training pipeline and use it to run experiments. Specifically you will:

1. **Rewrite the BPE tokenizer** as a byte-level tokenizer with an efficient (incrementally-updated) training loop, train it on the *full* TinyStories V2 corpus, and choose a vocabulary size on the basis of your own measurements.
2. **Build a proper training setup**: AdamW, a warmup + cosine learning-rate schedule, gradient clipping, bf16 mixed precision, and checkpointing.
3. **Implement efficient inference**: a KV cache, plus temperature / top-k / top-p sampling.
4. **Evaluate properly**, using perplexity and bits per character.
5. **Run a set of controlled experiments** — a learning-rate sweep, architecture ablations, and a vocabulary-size study — under a fixed compute budget, and report the results.

You should develop and debug everything on CPU (your laptop) and/or a free Colab GPU. You will then be given a limited allocation of **L4 GPU time** for your experiments and final training runs (Section 9). Managing that budget is part of the assignment.

Starting point. You may use the Tutorial 1 solution notebook as a starting point and copy code from it directly. Everything beyond what the tutorial already gives you must be your own work, subject to the AI policy in Section 12.

Reference. The design of this assignment follows Stanford CS336 *Assignment 1*, at a reduced scope. You may read the CS336 handout for background, but you may **not** consult or copy from any CS336 solution repository or any other third-party implementation. See Section 12.

1.1 What you may and may not use

The focus of this assignment is on the tokenizer, the inference machinery, and the experiments. Standard PyTorch building blocks are allowed, but any library that hands you a Transformer or a BPE tokenizer is not.

Allowed. `torch`, `numpy`, `matplotlib`, `regex`, `tqdm`, and the standard library. Within PyTorch you may anything except for the full layer implementaions listed below.

Not allowed. High-level architecture layers and pre-built tokenizers, specifically: `nn.Transformer`, `nn.TransformerEncoder`, `nn.TransformerDecoder`, `nn.TransformerEncoderLayer`, `nn.TransformerDecoderLayer`, `nn.MultiheadAttention`, `nn.RMSNorm`; anything from HuggingFace (`transformers`, `tokenizers`, `datasets`, `accelerate`); `sentencepiece`, `tiktoken`; and any third-party language-model implementation (nanoGPT, minGPT, llama2.c, nanochat, …).

You must write yourself (or reuse your own Tutorial 1 code for): BPE training, encoding and decoding; RMSNorm; SwiGLU; RoPE; multi-head causal self-attention and the transformer block; the KV cache; the data loader; the training loop; the evaluation metrics; and the sampling code.

1.2 Core and extension work

For each part of the assignment, a number of analysis questions are given. However, you are not expected to answer all of the questions, there is more analysis available here than anyone is expected to complete. You need to complete at least some of the analyses in each section, with the marks allocation as a guideline. Your answers will be assessed for completeness and depth of the analysis.

---

2. Dataset

We use **TinyStories V2 (GPT-4 generated)**, the same corpus as the tutorial, but the full version this time.

File	Size	Documents
`TinyStoriesV2-GPT4-train.txt`	~2.2 GB	~2.7 M
`TinyStoriesV2-GPT4-valid.txt`	~22 MB	~27 K

The dataset will be posted on Amathuba (Resources → Datasets). Documents should be separated by the `<|endoftext|>` delimiter.

Use the provided validation file for all held-out reporting. Do not train on it. Reserve the **last 2,000 documents** of the validation file as a test set that you touch exactly once, for the final numbers in Question 18.

Everything held out is scored with the two metrics defined in **Section 6**: per-token **perplexity** and **bits per character (BPC)**. You will need both, because you will be comparing models whose tokenizers differ — read Section 6 before you design your experiments, since it constrains what you have to record during evaluation (in particular, the raw character count of the evaluated text).

---

3. Task 1: An efficient byte-level BPE tokenizer

3.1 Move to bytes

The tutorial tokenizer operated on Unicode *characters* and needed an `<|unk|>` token for anything unseen. Re-implement it as a **byte-level** BPE tokenizer:

- The initial vocabulary is the 256 possible byte values, plus the special token `<|endoftext|>`. There is no unknown token — every possible input is representable.
- Pre-tokenization splits text into pre-tokens; each pre-token is then represented as a sequence of bytes and merged. Use the GPT-2 pre-tokenizer regex (Appendix A) with the `regex` package, which keeps leading spaces attached to words. This replaces the `_` word-start marker from the tutorial.
- Special tokens are **hard boundaries**: strip them out and split on them before pre-tokenizing, so no merge ever crosses a document boundary. They are not counted in the merge statistics, and are emitted directly as their own ID at encoding time.
- `decode` must round-trip: `decode(encode(s)) == s` for arbitrary UTF-8 input. Note that decoding a *prefix* of a token sequence may cut a multi-byte character in half — handle this with `errors="replace"` rather than crashing.

Required interface:

```
def train_bpe(input_path: str, vocab_size: int,
              special_tokens: list[str]) -> tuple[dict[int, bytes],
list[tuple[bytes, bytes]]]:
    """Returns (vocab: id -> token bytes, merges: ordered list of (left,
right) byte pairs)."""

class BPETokenizer:
    def __init__(self, vocab, merges, special_tokens=None): ...
    @classmethod
    def from_files(cls, vocab_path, merges_path, special_tokens=None): ...
    def encode(self, text: str) -> list[int]: ...
    def encode_iterable(self, iterable) -> Iterator[int]: ...   # memory-
efficient streaming
    def decode(self, ids: list[int]) -> str: ...
```

Ties in merge frequency must be broken deterministically by taking the **lexicographically greatest** pair, so that your merges are reproducible.

3.2 Make it fast

The tutorial's `train_bpe` recounts every pair from scratch after every merge, which is why it takes ~10 s for a 1,500-token vocabulary on 5,000 documents and would take many hours on the full corpus.

Required: incremental merge counting. Maintain an index of pair counts and, after applying a merge, update only the counts of pairs that overlap the merged occurrences, rather than recounting the whole corpus. You will also want an index from each pair to the pre-tokens that contain it. This is the change that matters, and it is the fiddly one — the bookkeeping is *remove the affected pre-token's old pair counts, apply the merge, add its new pair counts*, and it is easy to get subtly wrong. Test it by checking that your optimised trainer produces byte-identical merges to the tutorial version on a small corpus.

Recommended: parallel pre-tokenization. Once merging is fast, pre-tokenization becomes the bottleneck. It is embarrassingly parallel: split the input file into chunks aligned to `<|endoftext|>`

boundaries and count pre-tokens with `multiprocessing`, then merge the counters. This is worth doing but is not required for full marks.

Encoding also needs to be fast enough to process 2.2 GB in reasonable time: cache the token IDs of each distinct pre-token, and look merges up by rank rather than scanning the merge list.

Target: BPE training on the full training file should complete in **well under an hour** on a standard Colab CPU runtime, within the ~12 GB of RAM available there — under 10 minutes if you parallelise pre-tokenization. If you cannot fit in memory, you may learn the merges from a documented random subsample of the training file (state the fraction in your report) — but you must always *encode* the full file.

3.3 Sanity checks

Write these as tests: round-trip on 100 random validation documents; a document containing `<|endoftext|>` encodes to exactly one occurrence of that ID; every ID fits in `uint16`; and your merges are identical across two runs on the same input.

3.4 Choosing a vocabulary size

This is a decision you can make **without spending any GPU time**. Train BPE at a range of vocabulary sizes (at least 1,000 / 2,000 / 4,000 / 8,000 / 16,000) on the validation file or a subsample of the training file, and measure the compression ratio (bytes per token, and equivalently characters per token) for each. Note that you can track the corpus token count incrementally during training almost for free — every merge reduces it by exactly the count of the merged pair — so this whole study costs a few minutes of CPU. Using a vocabulary that is too large uses the model parameter budget less effectively.

Use the resulting curve, together with the effect on the parameter count, to choose the vocabulary size for your main experiments, and justify it in Question 3.

3.5 Encode the corpus

With your chosen tokenizer, encode the full training file and the validation file to `uint16` NumPy arrays saved with `np.save`, so that training can `np.load(..., mmap_mode="r")` them without reading hundreds of millions of tokens into RAM. Do this **once**, on CPU or Colab, and keep the output — never re-tokenize inside a GPU job.

You will also need a second tokenizer, at a clearly different vocabulary size, for the study in Section 7.3. Encode the corpus with that one too.

3.6 Questions — Task 1

Q1. Report the wall-clock time for training your BPE tokenizer at your chosen vocabulary size on the full training file, the wall-clock time to encode the full training file, and the machine you ran them on. *(1–2 sentences.)*

Q2. Train BPE with `vocab_size=1000` on a fixed 10 MB slice of the training data using (a) the Tutorial 1 implementation and (b) your optimised implementation. Report both times and the speedup. *(A three-number table.)*

Q3. Plot compression ratio (bytes per token) against vocabulary size for the range in Section 3.4. State the vocabulary size you chose for your main experiments and the reason. *(One figure plus 1–2 sentences.)*

Q4. Report the longest token in your main vocabulary, and list the first five and last five merges learned. Does the progression make sense? *(A short list plus 1–2 sentences.)*

---

4. Task 2: Model and inference

4.1 Base model

Reuse your Tutorial 1 Transformer (pre-norm blocks, RMSNorm, SwiGLU, RoPE, QK-norm, untied LM head), with the following hyperparameters:

Hyperparameter	Value
`vocab_size`	your choice from §3.4 (reference: 4,000)
`context_length`	256
`n_layers`	4
`d_model`	512
`n_heads`	8
`d_ff`	1344 (≈ ⅔ · `d_model`, a multiple of 64)
`rope_theta`	10,000
`use_qk_norm`	True

This gives roughly 17 M parameters excluding the embedding and LM head, plus `2 × vocab_size × d_model` for those.

Your model must be configurable, from a single config object or CLI flags, to support the ablations in Section 7: `use_rmsnorm`, `use_rope`, and `ffn_type ∈ {"swiglu", "relu"}`.

You may use `F.scaled_dot_product_attention` in place of your own attention computation; on a GPU it will dispatch to a fused (FlashAttention) kernel and is meaningfully faster. Your surrounding multi-head machinery — projections, head reshaping, RoPE, QK-norm, masking — must still be yours.

4.2 KV cache

At generation time, the tutorial's `generate` re-runs the entire prefix through the model for every new token, which is $O(n^2)$ work per sequence for what should be $O(n)$. Implement a KV cache:

- Each attention layer keeps buffers of shape `(batch, n_heads, context_length, d_head)` for keys and values, plus the number of positions currently filled.
- On a cached step the model is given only the new token(s); each layer computes Q, K, V for those positions, appends K and V to the cache, and attends over the full cached range.

- RoPE must be applied at the *absolute* position of each new token, not at position 0 — this is the most common bug. Positions come from the cache length.
- A causal mask is not needed for single-token decoding steps (the new query attends to everything already in the cache), but your implementation must still be correct when several tokens are prefilled at once.
- Handle the case where the sequence reaches `context_length`. Simply stopping generation there is acceptable, as long as it is documented.

Structure this as a *prefill* step (the whole prompt, in one forward pass) followed by *decode* steps (one token at a time). Do not duplicate your attention implementation — the cached and uncached paths should share code.

Correctness check: with a fixed seed and greedy decoding, cached and uncached generation must produce *identical* token sequences for a 100-token continuation, and the logits must agree to within floating-point tolerance.

4.3 Sampling

Extend `generate` to support, in this order: temperature scaling, then top-k truncation, then top-p (nucleus) truncation, then sampling.

- **Temperature** `T`: divide logits by `T` before the softmax. `T = 0` means greedy.
- **Top-k**: keep only the `k` highest-probability tokens, renormalise, sample.
- **Top-p**: keep the smallest set of tokens whose cumulative probability is ≥ `p`, renormalise, sample. Always keep at least one token.

Signature:

```
@torch.no_grad()
def generate(model, tokenizer, prompt: str, max_new_tokens: int = 256,
             temperature: float = 1.0, top_k: int | None = None,
             top_p: float | None = None, seed: int | None = None,
             use_cache: bool = True) -> str: ...
```

Generation stops at `<|endoftext|>` or `max_new_tokens`.

4.4 Questions — Task 2

Q5. Report your model's parameter count split into embedding, LM head, and non-embedding parameters, and confirm that your SwiGLU and ReLU FFN variants (§7.2) match in total parameters. *(A small table.)*

Q6. Give the evidence that your KV cache is correct: the maximum absolute logit difference between the cached and uncached paths, and whether greedy generations are identical. *(1–2 sentences.)*

Q7. Plot generation throughput (tokens/second) against the number of tokens generated, from 16 to 256, with and without the KV cache, on the same axes. State the speedup at 256 tokens. *(One figure plus one sentence.)*

---

5. Task 3: Training

You may use PyTorch's `AdamW` implementation, learning-rate schedulers, and `clip_grad_norm_`. Using them correctly still requires understanding what they do.

5.1 Optimiser

Use `torch.optim.AdamW` with β = (0.9, 0.95), ε = 1e-8, and weight decay 0.1 as starting values.

- AdamW's weight decay is **decoupled** — applied directly to the parameter rather than added to the gradient. `torch.optim.Adam(..., weight_decay=λ)` does the latter and is *not* the same optimiser; make sure you are using `AdamW`.
- Apply weight decay only to parameters with `ndim >= 2` (the weight matrices). Put RMSNorm gains and any biases in a separate parameter group with `weight_decay=0.0`. Decaying a normalisation gain towards zero is not a regulariser, it is a bug.

5.2 Learning-rate schedule

Use linear warmup followed by cosine decay: `torch.optim.lr_scheduler.LinearLR` chained with `CosineAnnealingLR` via `SequentialLR`, or `LambdaLR` with a function you write. Start from ~200 warmup steps and cosine decay to `0.1 × lr_max`.

Set the cosine period to **the actual length of the run**, so decay finishes exactly at the last step. If you shorten a run for a sweep, shorten the schedule too — otherwise you are comparing learning rates at different points on their schedules, and the comparison is meaningless.

Step the scheduler once per optimiser step, after `optimizer.step()`.

5.3 Gradient clipping

Call `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` after `backward()` and before `optimizer.step()`. It returns the **pre-clipping** total gradient norm — log it every step. It is one of the most informative diagnostics you have: a run about to diverge announces itself in the gradient norm well before the loss curve turns.

5.4 bf16 mixed precision

Train in mixed precision on GPU. Keep master weights, optimiser state, and the parameter update in **fp32**; run the forward and backward passes under

```
with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
    ...
```

Things to think about and to justify in your report:

- Which operations does autocast keep in fp32, and why does that matter for the softmax, the RMSNorm reciprocal square root, and the final cross-entropy?
- bf16 is only available on CUDA here — your code must fall back to fp32 on CPU and MPS.

5.5 Training loop

Requirements:

- **Data loader.** Sample batches of `(batch_size, context_length)` inputs with targets shifted by one, from a memory-mapped `uint16` array. Do not load the array into RAM, and do not read from disk inside the critical path of the step.
- **Checkpointing.** Save model, optimiser state, scheduler state, step count, RNG state, and config; support `--resume` to restart mid-run from a checkpoint. **This is not optional** — GPU jobs are queued and may be interrupted, and a run you cannot resume is a run you have to pay for twice.
- **Logging.** Append one row per evaluation to a JSONL or CSV file, containing at least: step, wall-clock seconds, tokens processed, learning rate, train loss, validation loss, pre-clip gradient norm. Every plot in your report must be generated from these logs by a script in your repo.
- **Validation** on a fixed set of batches (same seed every time) so curves are comparable across runs.
- **Determinism.** Seed `torch`, `numpy` and `random` from a single `--seed` flag.
- **Config.** Every hyperparameter must be settable from the command line or a config file. You will be launching a large number of runs; you do not want to be editing source code between them.

5.6 Before you spend GPU hours

Confirm on CPU or Colab first:

- Tests pass: BPE round-trip and determinism; KV cache versus no cache.
- The model **overfits a single batch** to near-zero loss within a few hundred steps. If it cannot, the bug is in your model or your loss, not your hyperparameters.
- The loss at initialisation is close to `ln(vocab_size)` (≈ 8.29 for a vocabulary of 4,000).
- A 50-step run resumed from a step-25 checkpoint reproduces the uninterrupted loss trajectory.
- A short run (e.g. 200 steps at `batch_size=8`) trains without NaNs and the loss goes down.

5.7 Questions — Task 3

Q8. Report the mean step time for your standard configuration in fp32 and in bf16 autocast. A short run (a few hundred steps) is enough for the timing. Which parts of your forward pass remain in fp32 under autocast, and why is that necessary? *(Two numbers plus 1–2 sentences.)*

Q9. Run your best learning rate with and without warmup, all else fixed, and plot both validation curves. Report the pre-clipping gradient norm at step 1, step 50, and the final step for each run. *(One figure plus a small table.)*

---

6. Evaluation metrics

Report **both** of the following on held-out data.

Perplexity. With N tokens $x_1, \ldots, x_N$ under your tokenizer,

$$\mathrm{PPL} = \exp\left(\frac{1}{N}\sum_{i=1}^{N} -\log p(x_i \mid x_{<i})\right) = \exp(\mathcal{L})$$

where $\mathcal{L}$ is exactly your mean cross-entropy loss in nats. Perplexity is the effective number of equally-likely tokens the model is choosing between at each step.

Bits per character (BPC). Perplexity depends on the tokenizer. A model with a 1,000-token vocabulary predicts *more, individually easier* tokens per document than one with a 16,000-token vocabulary, so their perplexities are not comparable — the two numbers are not even counting the same events. BPC normalises by the underlying text instead of by tokens:

$$\mathrm{BPC} = \frac{\sum_{i=1}^{N} -\log_2 p(x_i \mid x_{<i})}{C} = \frac{N \cdot \mathcal{L}}{C \cdot \ln 2}$$

where C is the number of **characters in the original text**, counted before tokenization. This is the average number of bits an optimal compressor using your model would need per character of text — a property of the model and the text, but not of the tokenization. Any two of your models can be compared on BPC, whatever their vocabularies.

Notes:

- N/C is the reciprocal of your tokenizer's compression ratio in characters per token, so BPC = loss ÷ (ln 2 × characters-per-token). You already measured that ratio in Section 3.4.
- Count C over exactly the text you evaluated on, and be consistent about whether the `<|endoftext|>` delimiter is included. State your choice.
- Evaluate over whole non-overlapping windows of `context_length`, and be aware that this under-counts context for tokens near the start of each window. Use the same procedure for every model you compare.

Indicative target (guidance, not a marking threshold): If you are well above 1.0 BPC after a full standard run, suspect a bug rather than a hyperparameter.

---

7. Task 4: Experiments

All experiments use the **standard run configuration** unless stated otherwise:

Setting	Value
Tokenizer	byte-level BPE, your chosen vocabulary size
Model	as in Section 4.1
Batch size	32 sequences × 256 tokens
Steps	5,000
Optimiser	AdamW, β = (0.9, 0.95), ε = 1e-8, weight decay 0.1 on matrices only
Schedule	200 warmup steps, cosine to 0.1 · lr_max, period = run length
Gradient clipping	max norm 1.0
Precision	bf16 autocast on GPU
Seed	fixed, the same for every run

These are *starting values*; you have to find the learning rate in particular.

When you vary one thing, hold everything else — including the seed, the number of tokens processed, and the validation batches — fixed. A comparison in which two things changed is not informative. If you have to change a second thing (for example, re-tuning the learning rate to get an ablation to train at all), say so explicitly and report both variants.

7.1 Learning-rate sweep

Sweep at least **five** peak learning rates spanning at least two orders of magnitude (a reasonable starting grid: 1e-4, 3e-4, 1e-3, 3e-3, 1e-2), and include **at least one run that diverges**. To save budget you may run the sweep at a reduced step count, as long as you set the cosine period to match. Then train the standard configuration at your best learning rate — this is your **baseline run**, the comparison point for everything below.

Kill diverged runs as soon as you can see they have diverged. There is nothing to learn from the remaining 4,800 steps and the budget is not free.

7.2 Ablations

Each ablation is one standard run, compared against your baseline. **Run at least two of the three**; the third is an extension.

1. **No RMSNorm.** Remove every RMSNorm from the blocks and the final norm. Train at your best learning rate; if it diverges, find a learning rate at which it does train, and report both.
2. **No positional information (NoPE).** Remove RoPE entirely. Everything else unchanged.
3. **ReLU instead of SwiGLU.** Replace $\mathrm{FFN}(x) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x)$ with $\mathrm{FFN}(x) = W_2\,\mathrm{ReLU}(W_1 x)$, and set $d_{ff} = 4d_{model} = 2048$ so the parameter counts match to within 2% (three matrices at 1344 versus two at 2048).

Your model should be configurable so that all three are a command-line flag away — the marginal cost of the third is then one queued job, and it is worth running if your budget allows.

7.3 Vocabulary size

Train one standard run with your second tokenizer from Section 3.5, at a clearly different vocabulary size (for example 1,000 if your main choice was 4,000), keeping every other setting the same. Note that this changes the model's parameter count as well as the tokenization — which is part of what makes the comparison interesting, and part of what makes it tricky to interpret.

7.4 Final model

Train your best configuration for roughly **four times** the standard step count, with the cosine period set to the full run length. This is the model you evaluate on the test set and generate from in Section 8.

7.5 Questions — Task 4

For questions Q11–Q13, answer the ones corresponding to the ablations you ran.

Q10. Plot validation loss against steps for your learning-rate sweep, all runs on one figure, including the divergent run. State your best learning rate, the lowest learning rate at which training diverged, and what

the gap between the two implies about how you should search for a learning rate on a new model. *(One figure plus 1–2 sentences.)*

Q11. Plot validation loss for the no-RMSNorm model at your best learning rate and at a reduced learning rate, against the baseline. What does RMSNorm appear to be doing? *(One figure plus 1–2 sentences.)*

Q12. Plot NoPE against your RoPE baseline. What is the size of the gap, and does it surprise you given that the model is causal? *(One figure plus 1–2 sentences.)*

Q13. Plot SwiGLU against the parameter-matched ReLU FFN and report the final validation loss gap. *(One figure plus 1–2 sentences.)*

Q14. Put the gaps from your two completed ablations side by side with the gap between your best and second-best learning rate from Q10. Which of these design choices mattered most at this scale? *(A four-row table plus 1–2 sentences.)*

Q15. Report perplexity, BPC, and total parameter count for your two vocabulary-size runs. Which model is better, and how does the parameter-count difference impact the comparison? *(A table plus 2–3 sentences.)*

Q16. Report the GPU-hours you used, split into development/debugging, the learning-rate sweep, the ablations, the vocabulary study, and the final model. If your budget were doubled, which single hyperparameter would you change, and which measurement from your own runs supports that? *(A table plus 1–2 sentences.)*

Q17. For your NoPE and RoPE models, plot mean validation loss as a function of position within the context window (bucketed, e.g. positions 0–31, 32–63, …, 224–255). What does the position-wise plot show that the aggregate curve does not? *(One figure plus 1–2 sentences.)*

---

8. Task 5: Final model and generation

Evaluate your final model from Section 7.4 on validation and — once only — on the held-out test documents from Section 2. Then generate from it.

8.1 Questions — Task 5

Q18. Report loss, perplexity, and BPC for your final model on validation and on test. *(A small table.)*

Q19. Generate 256 tokens from your final model with the prompt `"Once upon a time, "` under three decoding settings of your choice, at least one using top-p and one using top-k. Label each sample with its settings, and say in one sentence which you would ship and why. *(Three samples plus one sentence.)*

Q20. Give one decoding setting that produces visibly degenerate output (repetition loops, incoherence), and separately identify one systematic failure mode of the *model* that is not a decoding artefact. Quote at most three lines of generated text as evidence for each. *(Two short excerpts plus 1–2 sentences each.)*

---

9. Compute budget and GPU access

9.1 Phases

Phase 1 — develop on CPU and free Colab (now until you have GPU access). Everything in Sections 3, 4 and 5 can be written and tested without a dedicated GPU. Tokenizer training, the vocabulary-size study, and corpus encoding are CPU jobs and should be finished in this phase, with the results saved. Use a small debug configuration (`n_layers=2`, `d_model=128`, `batch_size=8`, a few hundred steps, the *validation* file as training data) to shake out bugs; a free Colab T4 is more than enough. **Do not start Phase 2 with untested code.**

Phase 2 — L4 GPU runs (12–18 August). Each student receives approximately **70 hours of NVIDIA L4 GPU time**, run as **queued jobs**. Access details, the job submission mechanism, and any per-job wall-clock limit will be announced on Amathuba. Because jobs are queued, throughput is not instantaneous: submit early, and always checkpoint.

> GPU access closes on **Tuesday 18 August**, the day before the deadline, so that you have a full day to write up. Anything still queued after that will not run.

9.2 Budgeting

These are the runs this assignment requires:

Experiment	Runs	Notes
Learning-rate sweep (§7.1)	≥ 5	may be shortened; one is expected to diverge early
Baseline (§7.1)	1	standard configuration at your best learning rate
Precision timing (Q8)	1 short	a few hundred steps in fp32, for step time only
Ablations (§7.2)	2–3	two required, third an extension; no-RMSNorm may need a second learning rate
Vocabulary size (§7.3)	1	
Final model (§7.4)	1	roughly 4× the standard step count
Extensions (Q9, Q17)	0–2	only if the core is finished and budget remains

Work out what this costs before you start. In your first GPU job, time 100 steps after a short warmup, record tokens per second, and use that number to plan the whole assignment. Report the figure in your experiment log. Do not assume the answer — it depends on your data loader, your batch size, and whether you used `torch.compile`.

A well-organised student should finish comfortably inside the allocation. The margin exists for the runs you get wrong, and you will get some wrong. Things that will waste your budget: recomputing the tokenizer on a GPU node; evaluating validation loss every 10 steps on 100 batches; a data loader that reads from disk in the critical path; forgetting `model.eval()` and `torch.no_grad()` during evaluation; letting a diverged run finish; not checkpointing.

---

10. Deliverables

Submit **one ZIP archive** to Amathuba, named `<studentnumber>_A1.zip`, containing the following.

10.1 Code repository (with git history)

Your code must be a **git repository**, submitted with its `.git` directory intact.

- `AGENTS.md`, provided with this assignment, must be present **unmodified in your first commit** and in every commit thereafter.
- Commit incrementally as you work — we expect **at least 15 commits spread over multiple days**, with messages that describe what changed. A repository whose entire history is one commit on 19 August will lose marks and may be referred for an oral assessment.
- Do **not** commit the dataset, the encoded `.npy` token arrays, or model checkpoints. Add a `.gitignore`. Do commit your tokenizer vocab/merges files and your experiment logs — they are small.

Suggested layout:

```
csc3043s_a1/
  AGENTS.md
  README.md
  requirements.txt
  .gitignore
  src/
    tokenizer.py      # train_bpe, BPETokenizer
    model.py          # RMSNorm, SwiGLU, RoPE, attention, block,
TransformerLM, KV cache
    data.py           # memmap loader, batching
    train.py          # training loop, optimiser/schedule setup,
checkpointing, logging, CLI
    evaluate.py       # perplexity, BPC
    generate.py       # sampling + KV-cache generation
  scripts/
    encode_corpus.py
    vocab_study.py
    run_experiments.sh   # the exact commands for every run in the report
    make_plots.py
  tests/
    test_tokenizer.py test_kv_cache.py test_model.py
  logs/               # one JSONL/CSV per run
  configs/            # one config per experiment (recommended)
```

`README.md` must state how to install dependencies, how to reproduce each experiment (exact commands), which Python and PyTorch versions you used, and where each figure in the report comes from.

10.2 Report (PDF)

At most **8 pages** excluding appendices. Structure it as the numbered answers Q1–Q20, in order, grouped by part, with a short (≤ 1 page) introduction describing your implementation choices and anything non-obvious you did.

Answer format matters. Each question asks for either (a) one to two sentences, (b) a small table, (c) a figure, or (d) a short sample of generated text. Answers that ramble beyond what is asked will lose marks. Every number you quote must be traceable to a log file in your repository.

Figures: label axes including units, include a legend, and use the same axis limits when comparing curves.

10.3 Experiment log

A single table (an appendix to the report, or `EXPERIMENTS.md` in the repo) listing every run you launched: run name, configuration, step count, final validation loss, wall-clock time, GPU-hours, and a one-line note on what you learned. Include the runs that failed — they are evidence of the process and they cost you budget.

10.4 Generated samples

A plain-text file `samples.txt` containing the generations from Questions 19 and 20, with the exact decoding settings labelled above each sample.

---

11. Marking

Total: 60 marks. Analysis marks are awarded for a task's answers as a whole, not question by question.

Task	Item	Type	Marks
Tokenizer (§3)	Byte-level BPE training: correctness, special tokens, determinism	impl	3
	Efficiency: incremental merge counting	impl	3
	Encoding/decoding, streaming, corpus preparation, sanity checks	impl	3
	Answers to Q1–Q4	analysis	3
Model and inference (§4)	KV cache: correctness, prefill/decode structure, RoPE positions	impl	3
	Temperature, top-k and top-p sampling	impl	3
	Answers to Q5–Q7	analysis	3
Training (§5)	Training loop, memory-mapped loader, checkpoint/resume, logging	impl	3
	Optimiser, parameter groups, schedule, clipping and bf16 set up correctly	impl	3
	Answers to Q8–Q9	analysis	3
Evaluation (§6)	Perplexity and BPC correct; consistent protocol; test set touched once	impl	3

Task	Item	Type	Marks
Experiments (§7)	Experimental hygiene: controlled comparisons, reproducible scripts, complete logs	impl	3
	Required runs executed and logged	impl	3
	Answers to Q10–Q17	analysis	9
Final model (§8)	Final model trained and evaluated on the held-out test set	impl	3
	Answers to Q18–Q20	analysis	3
Engineering practice	Code quality: structure, naming, tests, README, reproducibility	impl	3
	Git history: incremental, meaningful commits, `AGENTS.md` from the first commit	impl	3
Total			**60**

Three consequences worth being explicit about:

- **You are not required to answer every question.** As a guideline, aim to answer around two thirds of the overall questions.
- **A run that failed is still evidence.** If an experiment did not complete — the job was killed, the budget ran out, the ablation diverged and you could not stabilise it — say so plainly, show what you have, and state what you would have run. That is worth marks. A fabricated result is worth zero and is treated as academic dishonesty.

Report quality — structure, figure labelling, concision, traceability — is assessed as part of the analysis marks rather than separately. Answers that ramble well beyond what a question asks will pull the analysis mark down.

---

12. Academic integrity and AI usage

The Department's academic dishonesty policy and the AI usage rules in the *Notes to Students* apply in full. In addition, for this assignment:

AGENTS.md is mandatory. When you use an AI coding agent (Claude Code, Cursor, Copilot Agent, Codex, etc.) **or** a chat interface (ChatGPT, Claude, Gemini, etc.) on this assignment, you must place the provided `AGENTS.md` file in the assistant's context: for agents, keep it at the repository root where the tool reads it automatically; for chat interfaces, paste or attach it at the start of every conversation. It constrains the assistant to explanation, debugging guidance, and code review, and forbids it from writing solution code.

In short: AI may help you *understand* and *debug*, and may write code that visualises data your experiments have already produced. It may not write your tokenizer, model, KV cache, training loop, experiment scripts, or analysis. It may not design your experiments or interpret your results for you. Report prose may be edited for grammar by AI tools; the analysis must be yours.

Declaration. Include a short *AI usage declaration* in your report (appendix, not counted in the page limit) stating which tools you used, for what, and confirming that `AGENTS.md` was in context. A declaration of "none" is perfectly acceptable.

Third-party code. You may not consult or copy from CS336 solution repositories, nanoGPT/minGPT/llama2.c/nanochat, HuggingFace implementations, or any other existing Transformer or BPE implementation. You may consult the official PyTorch documentation, the lecture slides, the tutorial notebooks, and the papers cited in Appendix B.

Oral assessment. A sample of submissions may be selected for a short oral assessment in which you will be asked to explain your own code and results. Marks may be adjusted based on that assessment.

Appendix A: GPT-2 pre-tokenizer regex

```
PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+
(?!\S)|\s+"""
```

Use the `regex` package (`pip install regex`), not the standard `re` module — the standard module does not support `\p{L}`. Apply it with `regex.finditer(PAT, chunk)`.

Appendix B: References

1. Eldan & Li (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?*
2. Sennrich, Haddow & Birch (2016). *Neural Machine Translation of Rare Words with Subword Units.*
3. Radford et al. (2019). *Language Models are Unsupervised Multitask Learners* (GPT-2).
4. Loshchilov & Hutter (2019). *Decoupled Weight Decay Regularization* (AdamW).
5. Shazeer (2020). *GLU Variants Improve Transformer.*
6. Su et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding.*
7. Zhang & Sennrich (2019). *Root Mean Square Layer Normalization.*
8. Holtzman et al. (2020). *The Curious Case of Neural Text Degeneration* (top-p sampling).
9. Kazemnejad et al. (2023). *The Impact of Positional Encoding on Length Generalization in Transformers* (NoPE).